Stub Mock y cobertura JaCoCo

Universidad Espíritu Santo · UCOM0310 · Semana 7
Actividad formativa 2 · 30 de septiembre de 2026

Estudiante: Jordy Alessandro Fajardo Figueroa
Carrera: Ingeniería en Ciencias de la Computación
Materia: Diseño de Software

Repositorio y rama correctos

La consigna pide partir del resultado del laboratorio 1, conservar sus pruebas y separar el trabajo en una rama. Se continúa UEES-Formativa1-Semana7 en test/lab2-dobles-cobertura. No se necesita otro repositorio ni un Pull Request para esta actividad. La rama se publica sin fusionar; main conserva la actividad 1. Ae6 pertenece a un repositorio independiente.

Objetivo y resultado

Aislar los tres colaboradores de confirmar, comprobar sus efectos con Mockito e interpretar JaCoCo. La suite final conserva diez pruebas del laboratorio 1 y añade tres de confirmación: 13 pruebas, cero fallos, errores u omitidas. El servicio alcanza 100 % de líneas y ramas; el dominio conserva huecos que se declaran en el análisis.

Diferencia práctica entre Stub y Mock

Doble	Papel	Qué demuestra
DisponibilidadClient	Stub mediante when y thenReturn	Se elige true o false sin consultar disponibilidad real
ReservaRepository	Mock mediante verify	Guardar una vez en el éxito y cero veces en el rechazo
Notificador	Mock mediante verify	Notificar una vez en el éxito y cero veces en el rechazo

Mockito crea los dobles, pero el papel lo determina su uso. Configurar una respuesta no es una aserción. El Stub proporciona datos controlados; los Mocks comprueban interacciones relevantes. Reserva es un objeto real para observar el estado; simular ese estado ocultaría precisamente el comportamiento que se quiere comprobar.

No se comprueba la infraestructura real: las llamadas al repositorio y al notificador se observan sobre dobles y no persisten ni envían mensajes externos. En confirmar, el servicio recibe tres colaboradores válidos; la estrategia de null del laboratorio 1 solo se conserva para sus métodos puros.

Tres escenarios y evidencia Mockito

Disponible: se configura true, se llama confirmar y se exige CONFIRMADA. Además, se verifica una sola llamada a guardar y enviarConfirmacion, ambas con la misma instancia de reserva.

```
when(disponibilidad.estaDisponible(reserva)).thenReturn(true);
servicio.confirmar(reserva);
assertEquals(EstadoReserva.CONFIRMADA, reserva.getEstado());
verify(repository, times(1)).guardar(same(reserva));
verify(notificador, times(1)).enviarConfirmacion(same(reserva));
```

No disponible: se configura false. La prueba exige IllegalStateException con Horario no disponible, conserva PENDIENTE y comprueba que no se guarde ni notifique con ningún argumento.

```
when(disponibilidad.estaDisponible(reserva)).thenReturn(false);
assertThrows(IllegalStateException.class,
        () -> servicio.confirmar(reserva));
verify(repository, never()).guardar(any());
verify(notificador, never()).enviarConfirmacion(any());
```

Reserva nula: no se configura una respuesta porque no debe consultarse disponibilidad. La prueba exige IllegalArgumentException con Reserva obligatoria y cero interacciones con los tres colaboradores.

```
assertThrows(IllegalArgumentException.class,
        () -> servicio.confirmar(null));
verify(disponibilidad, never()).estaDisponible(any());
verify(repository, never()).guardar(any());
verify(notificador, never()).enviarConfirmacion(any());
verifyNoInteractions(disponibilidad, repository, notificador);
```

times(1) exige una llamada; never exige cero. any() permite comprobar la operación para cualquier argumento; no representa un dato de negocio. same(reserva) comprueba identidad. verifyNoInteractions refuerza el rechazo de toda interacción, incluso con argumentos null. Los fragmentos resumen las verificaciones; el archivo Java también comprueba mensajes y el estado PENDIENTE.

Resultado de ejecución

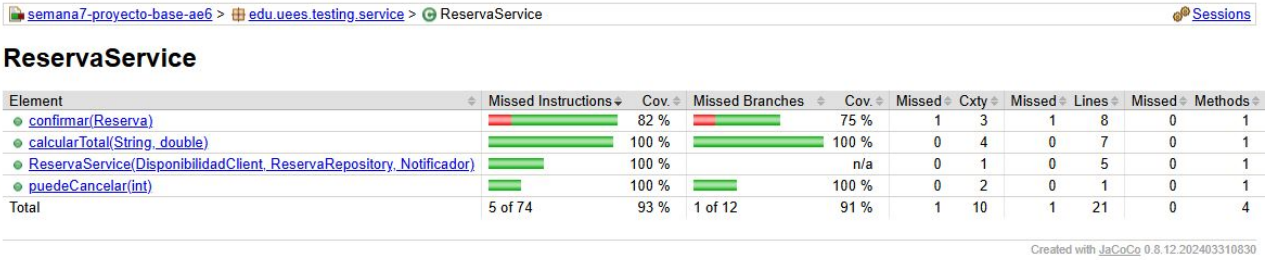
Java 21.0.12.1 y Maven 3.9.16. Comando: mvn -B clean test. Resultado: ConfirmacionLab2Test ejecutó 3 pruebas y ReservaServiceTest 10; total 13, 0 fallos, 0 errores, 0 omitidas, BUILD SUCCESS. El log 07_ejecucion_final.txt respalda estos valores. El XML 09_junit_confirmacion.xml conserva nombres y resultados Surefire, omitiendo propiedades del entorno y mensajes de consola.

Un verify incorrecto haría fallar su testcase. El resultado verde indica que las aserciones y verificaciones pasaron; no es una traza de integración ni prueba que un proveedor externo funcione.

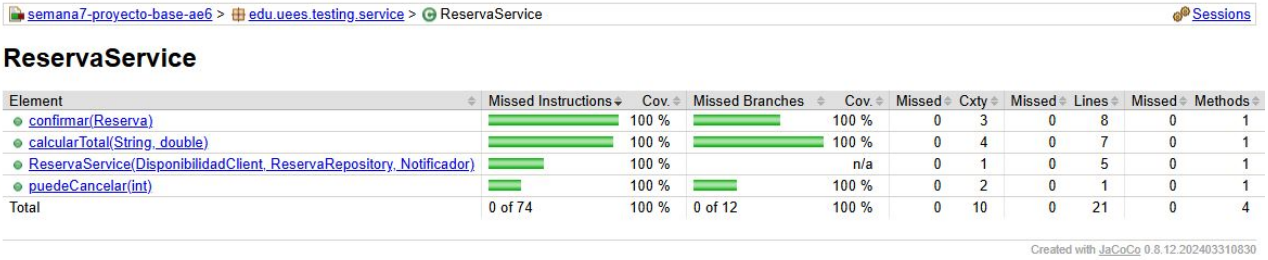
Comparación real de cobertura

ReservaService	Base lab 1	Dos escenarios	Final con null
Pruebas totales	10	12	13
Líneas	13/21 = 61,90 %	20/21 = 95,24 %	21/21 = 100 %
Ramas	8/12 = 66,67 %	11/12 = 91,67 %	12/12 = 100 %
Instrucciones	46/74 = 62,16 %	69/74 = 93,24 %	74/74 = 100 %

Antes de la mejora: captura del HTML real. confirmar conservaba una línea sin ejecutar y una rama pendiente: el rechazo de reserva nula. Su cobertura era 7/8 líneas, 3/4 ramas y 23/28 instrucciones.



Después: se añadió reservaNulaNoConsultaDependencias a partir del hueco observado. confirmar cubre 8/8 líneas y 4/4 ramas. ReservaService queda con cero líneas y ramas perdidas.



Cómo leer las columnas sin confundirlas

La primera Cov. corresponde a instrucciones; la segunda a ramas. Missed junto a Lines indica las líneas perdidas y Lines el total: cobertura de líneas = (total - perdidas) / total. El HTML muestra porcentajes enteros truncados, mientras que la tabla calcula dos decimales desde los CSV. Por ejemplo, 11/12 ramas aparece como 91 % en el HTML y equivale a 91,67 %.

Al entrar al código fuente, rojo señala instrucciones no ejecutadas, amarillo cobertura parcial y verde instrucciones ejecutadas. Los diamantes muestran decisiones. La captura 06_hueco_reserva_nula.png registra la condición parcialmente cubierta y su excepción pendiente; 11_codigo_final.png registra el resultado final.

Qué todavía no está protegido

El proyecto completo tiene 34/39 líneas (87,18 %) y 15/18 ramas (83,33 %). Reserva conserva 9/14 líneas y 3/6 ramas: faltan validaciones de ID, tipo nulo, getId, getTipo y cancelar. El 100 % de ReservaService no debe presentarse como 100 % del proyecto.

No se prueban integraciones, transacciones, concurrencia ni recuperación si guardar o notificar fallan. Si guardar falla, la reserva ya fue confirmada en memoria; definir la recuperación requiere un contrato explícito. Tampoco se acuerdan redondeo, NaN e infinito para importes.

Cobertura no garantiza corrección. Ejecutar confirmar sin comparar el estado o verificar sus efectos podría cubrir el código sin detectar omisiones. Las tasas VIP y ESTUDIANTE también podrían estar equivocadas conservando las mismas ramas: los esperados 85 y 90 del laboratorio 1 son los que detectan ese error. La prueba inicial assertTrue(true) no protege una regla.

Guion para mostrar la evidencia

1. Mostrar git branch y la rama del laboratorio 2; señalar que las diez pruebas anteriores se conservan.
2. Abrir ConfirmacionLab2Test.java: explicar el when del Stub, el estado y los verify del éxito.
3. Mostrar false y null: explicar excepciones, estado intacto y never con any.
4. Abrir el log final y señalar 3 + 10 = 13 pruebas y BUILD SUCCESS.
5. Comparar las capturas antes/después y relacionar el hueco con la nueva prueba.
6. Mostrar git log --oneline --decorate y los commits descriptivos.

Git y reproducción

El primer commit añadió disponibilidad y rechazo con dobles; el segundo añadió null tras revisar JaCoCo y conservó la medición previa; el tercero incorpora resultados finales y documentación. La línea base del laboratorio 1 es e134cb8. Los archivos productivos y sus pruebas previas permanecen sin cambios.

```
git switch test/lab2-dobles-cobertura
mvn -version
mvn clean test
git log --oneline --decorate
```

Maven debe usar Java 21. Abrir target/site/jacoco/index.html y entrar al paquete del servicio. target y tmp no se versionan. El identificador Maven conserva el nombre del proyecto base Ae6, pero el repositorio utilizado es el de las formativas. Las advertencias de agente dinámico Mockito no son fallos de pruebas.

Rama de entrega en GitHub. Guía detallada: docs/lab2/01_GUIA_EVIDENCIAS.md. Análisis: docs/02_ANALISIS_COBERTURA_PLANTILLA.md. Logs, CSV, XML y capturas: docs/lab2/evidencias/.

Reflexión y uso de IA

El Stub permite elegir la respuesta externa; el Mock permite exigir o prohibir efectos observables. JaCoCo orientó una mejora concreta, pero las aserciones son las que validan la regla. Codex asistió en implementación, ejecución, capturas y documentación; los resultados provienen de ejecuciones reales. La revisión asistida no es aprobación independiente y el estudiante debe comprender la evidencia antes de entregar.